0199. Binary Tree Right Side View

dium | Category: Trees | LeetCode: https://leetcode.com/problems/binary-tree-rig

Problem

- Return the values visible when looking at the tree from the right: the last node of each level, top to bottom.

Intuition

- Level-order BFS; the last node popped in each level is visible.
- DFS alternative: visit right child first and record the first node seen at each new depth.

Brute Force

- Full level-order traversal then take the last of each list: same complexity, more memory.

Optimal Approach

- BFS by level.
- For each level, record the value of the node at index size-1.

Worked Example

```
[1,2,3,null,5,null,4].

level [1]    -> 1
level [2,3]  -> 3
level [5,4]  -> 4
answer [1,3,4]
```

Java Solution

```java
public class BinaryTreeRightSideView {
    public List<Integer> rightSideView(TreeNode root) {
        List<Integer> res = new ArrayList<>();
        if (root == null) return res;
        Deque<TreeNode> q = new ArrayDeque<>(List.of(root));
        while (!q.isEmpty()) {
            int size = q.size();
            for (int i = 0; i < size; i++) {
                TreeNode n = q.poll();
                if (i == size - 1) res.add(n.val);
                if (n.left != null) q.add(n.left);
                if (n.right != null) q.add(n.right);
            }
        }
        return res;
    }
}
```

Complexity

- Time: O(n). Space: O(width).

Pitfalls

- A left node can be visible if the right side is shorter ([1,2] → [1,2]).